

Employee Raise Issue — ServiceNow Project Documentation

1. Problem Statement

In a large enterprise organization, employees frequently face workplace-related issues such as **payroll discrepancies, access problems, infrastructure requests, and HR policy clarifications**.

Currently, employees raise these issues through **emails or informal communication channels**, which creates several challenges:

- **Lack of standardized data capture** — Employee issues are reported in inconsistent formats.
- **Delayed issue resolution** — Support and HR teams spend additional time understanding and processing requests.
- **No visibility into issue status** — Employees cannot easily track whether their issue is pending, assigned, or resolved.
- **Manual effort for HR and support teams** — Teams must manually collect information, assign issues, and communicate updates.
- **Inconsistent prioritization and assignment** — Issues may not be routed to the appropriate team or prioritized consistently.

Proposed Solution

To address these challenges, a **ServiceNow Employee Raise Issue application** will be developed.

The solution will provide employees with a centralized **Service Portal interface** where they can raise workplace-related issues using a standardized form.

The solution will use:

- A custom **Employee Raise Issue table**
- A **Record Producer** for submitting employee issues
- A **Service Portal Widget** to provide an easy-to-use interface
- **Business Rules** to enforce business logic and dependencies
- Standardized issue information and categorization
- Automated assignment and prioritization where applicable
- Issue status tracking and improved visibility

The overall goal is to provide a **structured, user-friendly, and scalable mechanism for employees to raise and track workplace issues**, while reducing manual effort for HR and support teams.

2. Project Implementation Phases

The project will be implemented through multiple phases:

The remaining phases will be added as the implementation progresses.

Phase Description

Phase 1 Create the **Employee Center custom scoped application**

Phase 2 Create the **Employee Raise Issue custom table** and configure required fields

Phase 3 Create **UI Policies and field dependencies** to control form behavior

Phase 4 Create the **Raise Employee Issue Record Producer** and map variables to table fields

Phase 5 Create the **Requesting Service Portal** and Employee Center Portal Page

Phase 6 Create **custom Service Portal widgets** and add them to the Employee Center Portal Page

Phase 7 Perform **end-to-end testing and validation** of the application

Phase 8 Conclusion — summarize the solution, benefits, and overall project outcome

Phase 1 — Create Custom Application

Objective

Create the foundational ServiceNow application that will contain the tables, modules, business logic, and other components required for the **Employee Raise Issue** solution.

Description

This phase involves creating a new custom ServiceNow application by defining its **name, scope, and core configuration**. It establishes the foundation for building the Employee Raise Issue functionality.

Steps to Create an Application

1. Navigate to **All → App Engine → ServiceNow Studio**.
2. Click **Create**.
3. Select **App** to create a new application.
4. Enter the following application details:
 - **Name:** Employee Center
 - **Scope:** Scoped
5. Click **Continue**.
6. Return to the ServiceNow home page.
7. Reload the instance if required.
8. Verify that **Employee Center** appears as the current application.

Expected Result

The **Employee Center** scoped application is successfully created and is available as the foundation for implementing the remaining components of the Employee Raise Issue solution.

Phase 2 — Create Employee Raise Issue Custom Table

Objective

Create a custom table to store and manage **Employee Raise Issue** records. The table will contain the necessary fields to capture employee information, issue classification, description, state, priority, and assignment details.

Description

This phase establishes the **data structure** required for the Employee Raise Issue application.

A custom **Employee Raise Issue** table is created, along with fields such as **Requester, Category, Subcategory, Short Description, State, Priority, and Assignment Group**.

These fields provide standardized data capture and will later be used by the **Record Producer, Service Portal Widget, and Business Rules**.

Subtask 1 — Create the Custom Table

Steps

1. Navigate to **All → System Definition → Tables**.
2. Click **New** in the top-right corner.
3. Enter the following details:
 - **Label:** Employee Raises Issue
 - **Name:** Automatically populated by ServiceNow.
4. Under **Controls**, enable:
 - **Auto number**
5. Save the form using the **Context Menu**.

Expected Result

The **Employee Raises Issue** custom table is successfully created with automatic numbering enabled.

Subtask 2 — Create Fields for the Table

After creating the table, configure the fields required to capture employee issue information.

Steps

1. Open the **Employee Raises Issue** table.
2. Under **Related Links**, click **Show Form**.

3. Open the **Context Menu**.

4. Navigate to:

Configure → Form Design

5. Use **Form Design** to create and configure the required fields.

Field Configuration

Field Label	Type	Notes
Number	Auto-Number	Automatically generated by ServiceNow
Requester	Reference	References the sys_user table
Category	Choice	Network, Hardware, Software, Access
Subcategory	Choice	VPN, Laptop, Server, Forgot Password
Short Description	String	Brief summary of the issue
State	Choice	New, In Progress, Resolved, Closed
Priority	Choice	Critical, High, Medium, Low
Assignment Group	Reference	References the sys_user_group table

Field Details

1. Number

- **Field Label:** Number
- **Type:** Auto-Number
- **Purpose:** Automatically generates a unique identifier for each Employee Raise Issue record.

Example:

ERI0001001

ERI0001002

ERI0001003

2. Requester

- **Field Label:** Requester
- **Type:** Reference
- **Reference Table:** sys_user

- **Purpose:** Identifies the employee who raised the issue.

This allows the issue to be associated with a specific ServiceNow user.

3. Category

- **Field Label:** Category
- **Type:** Choice
- **Choice Values:**
 - Network
 - Hardware
 - Software
 - Access

Purpose: Identifies the broad category of the employee's issue.

4. Subcategory

- **Field Label:** Subcategory
- **Type:** Choice
- **Choice Values:**
 - VPN
 - Laptop
 - Server
 - Forgot Password

Purpose: Provides more specific classification of the issue.

Dependency: Later Business Rules or Catalog Client Scripts can use **Category → Subcategory** dependencies to ensure that only relevant subcategories are available for a selected category.

5. Short Description

- **Field Label:** Short Description
- **Type:** String
- **Purpose:** Captures a brief summary of the employee's issue.

Example:

Unable to connect to company VPN

6. State

- **Field Label:** State
- **Type:** Choice
- **Choice Values:**
 - New
 - In Progress
 - Resolved
 - Closed

Purpose: Tracks the current lifecycle status of the issue.

7. Priority

- **Field Label:** Priority
- **Type:** Choice
- **Choice Values:**
 - Critical
 - High
 - Medium
 - Low

Purpose: Indicates the urgency and importance of the employee's issue.

Dependency: Priority can later be automatically determined using Business Rules based on factors such as **Category, issue type, or other business conditions**.

8. Assignment Group

- **Field Label:** Assignment Group
- **Type:** Reference
- **Reference Table:** sys_user_group
- **Purpose:** Identifies the support team responsible for handling the employee's issue.

For example:

- Network issue → Network Support
- Hardware issue → Hardware Support

- Software issue → Application Support
 - Access issue → Access Management
-

Expected Result

After completing this phase, the **Employee Raises Issue** custom table contains the required structure for storing employee issues.

The table can now capture:

Employee → Issue Category → Subcategory → Description → State → Priority → Assignment Group

This table will serve as the **backend data source** for the upcoming Record Producer and Service Portal implementation.

Phase 3 — Creating UI Policies and Field Dependencies

Objective

Configure **UI Policies** and **field dependencies** for the Employee Raise Issue form to dynamically control field behavior based on user input.

This phase ensures that fields can be made **mandatory, read-only, visible, or dependent on other fields** according to business requirements. This improves usability, enforces data accuracy, and prevents incorrect issue classification.

Subtask 1 — Make Short Description Mandatory When State Is New

Description

A UI Policy is created to make the **Short Description** field mandatory whenever the **State** of an Employee Raise Issue record is set to **New**.

Navigation Steps

1. Navigate to **All → UI Policies**.
2. Click **New** in the top-right corner.
3. Enter the following details:
 - **Table:** Employee Raise Issue
 - **Short Description:** When the state changes to New, Short Description is mandatory.
4. Under **Conditions / When to Apply**, configure:

Field Operator Value

State is New

5. Save the UI Policy.

6. Under the **Related Lists**, locate the **UI Policy Actions** section.
7. Click **New**.
8. Configure the UI Policy Action:

Field	Value
Field Name	Short Description
Mandatory	True
Visible	Leave unchanged
Read Only	Leave unchanged

9. Click **Submit**.
10. Return to the UI Policy and click **Update** if required.

Validation

1. Open the **Employee Raise Issue** form.
2. Reload the page.
3. Change **State** to **New**.
4. Verify that **Short Description** becomes mandatory.

Expected Result

Whenever the State is **New**, the **Short Description** field must be populated before the record can be submitted.

Subtask 2 — Make Number Field Read-Only

Description

A UI Policy is created to ensure that the automatically generated **Number** field cannot be manually modified by users.

Since the Number field is generated by ServiceNow through the auto-number configuration, it should remain **read-only**.

Navigation Steps

1. Navigate to **All → UI Policies**.
2. Click **New** in the top-right corner.
3. Enter the following details:
 - **Table:** Employee Raise Issue
 - **Short Description:** Number field will be read-only.

- **Condition:** No filter condition is required.
- 4. Save the UI Policy.
- 5. Under the **Related Lists**, locate the **UI Policy Actions** section.
- 6. Click **New**.
- 7. Configure the UI Policy Action:

Field	Value
Field Name	Number
Read Only	True
Mandatory	Leave unchanged
Visible	Leave unchanged

- 8. Click **Submit**.
- 9. Return to the UI Policy and click **Update** if required.

Expected Result

The **Number** field is displayed as **read-only**, preventing users from manually changing the automatically generated issue number.

Subtask 3 — Configure Category and Subcategory Dependency

Description

A field dependency is configured between **Category** and **Subcategory**.

The purpose is to ensure that the **Subcategory** field displays only the appropriate choice based on the selected **Category**.

Dependency Mapping

Category **Subcategory**

Network VPN

Hardware Laptop

Software Server

Access Forgot Password

For example, when the user selects **Network** as the Category, only **VPN** should be available as the corresponding Subcategory.

Navigation Steps

Step 1 — Open the Subcategory Dictionary

1. Open the **Employee Raise Issue** table form.
 2. Locate the **Subcategory** field.
 3. Right-click the **Subcategory** field.
 4. Select **Configure Dictionary**.
 5. A new dictionary form opens.
-

Step 2 — Open Advanced View

6. Under the **Related Links**, select **Advanced View**.
7. Locate the **Dependent field** configuration.
8. Set the dependent field to:

Category

This establishes the relationship:

Category

|

└─ Subcategory

Step 3 — Configure Choice Dependencies

9. Under the **Related Links**, locate the **Choices** section.
10. Configure the Subcategory choices with the appropriate backend values of the **Category** choices.
11. Associate each Subcategory choice with its corresponding Category.

Expected Result

The **Subcategory** field dynamically displays the appropriate choices based on the selected **Category**.

Phase 3 — Final Outcome

After completing all three subtasks, the Employee Raise Issue form now has dynamic behavior:

Configuration	Result
State = New	Short Description becomes mandatory
Number	Remains read-only
Category → Subcategory Subcategory choices are filtered according to Category	

This phase provides the **form-level validation and dynamic behavior** required before moving to the Record Producer and Service Portal implementation.

Phase 4 — Creating a Record Producer

Objective

Create a **Record Producer** that allows employees to submit an Employee Raise Issue through the ServiceNow Service Catalog.

The Record Producer provides a user-friendly form for collecting issue information and maps the submitted variables to fields in the **Employee Raise Issue** custom table.

Description

A Record Producer is configured to provide employees with a standardized form for raising workplace-related issues.

The Record Producer is connected to the **Employee Raise Issue** table, so when an employee submits the form, ServiceNow creates a corresponding record in the custom table.

The form captures important information such as:

- Requester
- Category
- Subcategory
- State
- Short Description

Subtask 1 — Create the Record Producer

Navigation Steps

1. Navigate to:

All → Service Catalog → Catalog Definition → Record Producers

2. Click **New** in the top-right corner.
3. Enter the following details:

Field	Value
Name	Raise Employee Issue
Table	Employee Raise Issue

What it will contain Description of the purpose of the Record Producer

4. In the **Accessibility** section, configure:
 - **Catalog:** Service Catalog
 - **Category:** Service
5. Add an appropriate image in the **Picture** field.
6. Enter the **Meta** name based on the Employee Raise Issue table name.
7. Save the form.

Expected Result

The **Raise Employee Issue** Record Producer is successfully created and is configured to create records in the **Employee Raise Issue** table.

Subtask 2 — Create Record Producer Variables

Description

Variables are created to collect information from the employee when they submit an issue.

These variables act as the input fields displayed to the user through the Record Producer.

Each variable is mapped to the corresponding field in the **Employee Raise Issue** table using the **Map to Field** option.

Navigation Steps

1. After saving the Record Producer, scroll to the **Related Lists**.
2. Locate the **Variables** section.
3. Click **New**.
4. Select the appropriate **Type** based on the corresponding Employee Raise Issue table field.
5. Enable the **Map to Field** option.
6. Select the appropriate table **Field**.
7. Set the variable **Order**.

Recommended order:

- 100
 - 200
 - 300
 - 400
 - 500
8. In the **Question** section, enter the question that should be displayed to the employee.

9. The **Name** field will be populated automatically based on the question.

10. Repeat these steps to create all five variables.

Variable Configuration

Variable / Question	Type	Reference / Values	Mapped Table Field
What is the Requester name?	Reference	sys_user	Requester
Which Category?	Choice	Network, Software, Hardware, Access	Category
What is Subcategory?	Choice	VPN, Laptop, Server, Forgot Password	Subcategory
Which State is this issue?	Choice	New, In Progress, Resolved, Closed	State
Short description	Single Line Text	String	Short Description

Variable 1 — Requester

Question:

What is the Requester name?

Configuration:

- **Type:** Reference
- **Reference Table:** sys_user
- **Map to Field:** Enabled
- **Table Field:** Requester
- **Order:** 100

Purpose

Identifies the employee who is raising the issue.

Variable 2 — Category

Question:

Which Category?

Configuration:

- **Type:** Choice
- **Choices:**
 - Network
 - Software
 - Hardware
 - Access
- **Map to Field:** Enabled
- **Table Field:** Category
- **Order:** 200

Purpose

Identifies the broad category of the employee's issue.

Variable 3 — Subcategory

Question:

What is Subcategory?

Configuration:

- **Type:** Choice
- **Choices:**
 - VPN
 - Laptop
 - Server
 - Forgot Password
- **Map to Field:** Enabled
- **Table Field:** Subcategory
- **Order:** 300

Purpose

Provides a more specific classification of the issue.

The Subcategory is related to the Category dependency configured in **Phase 3**.

Variable 4 — State

Question:

Which State is this issue?

Configuration:

- **Type:** Choice
- **Choices:**
 - New
 - In Progress
 - Resolved
 - Closed
- **Map to Field:** Enabled
- **Table Field:** State
- **Order:** 400

Purpose

Captures the current state of the Employee Raise Issue record.

Important: In a typical production implementation, newly raised issues would generally be created with the **New** state automatically rather than allowing employees to select lifecycle states such as Resolved or Closed. If your project specifically requires employee selection, keep the configuration as defined above.

Variable 5 — Short Description

Question:

Short description

Configuration:

- **Type:** Single Line Text
- **Data:** String
- **Map to Field:** Enabled
- **Table Field:** Short Description
- **Order:** 500

Purpose

Captures a brief summary of the employee's issue.

Example:

Unable to access company VPN

This mapping ensures that the information entered by the employee is stored in the appropriate fields of the custom table.

Expected Result

After completing Phase 4:

1. The **Raise Employee Issue** Record Producer is available in the Service Catalog.
2. Employees can open the Record Producer and enter their issue details.
3. The five variables collect standardized information.
4. Each variable is mapped to its corresponding field in the **Employee Raise Issue** table.
5. On submission, ServiceNow creates an **Employee Raise Issue** record containing the submitted information.
6. The **Category → Subcategory** dependency configured in Phase 3 can be applied to the Record Producer experience.

Phase 5 — Creating a New Service Portal

Objective

Create a dedicated **Service Portal** for employees to access the Employee Raise Issue functionality.

The portal provides the employee-facing interface through which users can access services, submit requests, and interact with the Employee Center application.

Description

In this phase, a new Service Portal named **Requesting Portal** is created and configured with the required homepage, knowledge page, login page, menu, theme, logo, and 404 page.

A separate portal page is also created to provide a dedicated page within the portal for the Employee Center functionality.

Subtask 1 — Create a New Service Portal

Navigation Steps

1. Navigate to:

All → Service Portal → Service Portal Configuration

2. Click **New** in the top-right corner.
3. Configure the Service Portal with the following details:

Configuration Value

Title	Requesting Portal
URL Suffix	employee_request
Homepage	ec_standard_home
KB Home Page	esc_knowledge_home
Login Page	login
Logo	Upload the required image
Main Menu	Employee Center Menu
404 Page	404
Theme	Employee Center Coral Theme

4. Verify that all required fields are configured correctly.
5. Save the Service Portal using the **Context Menu**.

Expected Result

The **Requesting Portal** is successfully created with the configured homepage, knowledge base page, login page, menu, theme, logo, and error page.

The portal can now serve as the employee-facing interface for the Employee Raise Issue application.

Subtask 2 — Create a New Employee Center Portal Page

Objective

Create a dedicated portal page that can be used to display Employee Center functionality and components within the newly created Service Portal.

Navigation Steps

1. Navigate to:

All → Service Portal

2. Select the **Pages** module.
3. Click **New** in the top-right corner.
4. Enter the following details:
 - **Title:** Employee Center Portal Page
 - **ID:** employee_center_portal_page
5. Open the **Context Menu**.

6. Click **Save**.

Expected Result

The **Employee Center Portal Page** is successfully created with the following configuration:

Title: Employee Center Portal Page

ID: employee_center_portal_page

This page can subsequently be configured with the required **Service Portal widgets and Employee Raise Issue functionality**.

Phase 5 — Expected Outcome

After completing this phase, the project has:

- A dedicated **Requesting Portal** for employees.
- A configured portal URL suffix: employee_request.
- A configured Employee Center homepage.
- A configured knowledge base homepage.
- A login page and 404 page.
- An Employee Center menu.
- An Employee Center theme and logo.
- A dedicated **Employee Center Portal Page** ready for widget configuration.

Phase 6 — Creating Widgets and Adding Them to the Page

Objective

Create custom **Service Portal widgets** and add them to the Employee Center Portal Page to provide employees with an easy-to-use interface for accessing the **Raise Employee Issue** Record Producer and other relevant information.

Description

This phase involves creating two custom Service Portal widgets and placing them on the Employee Center Portal Page.

The first widget provides a visual/interactive element, while the second widget provides navigation functionality to redirect employees to the **Raise Employee Issue Record Producer**.

The widgets are then positioned on the existing Employee Center Portal Page using the **Service Portal Designer**.

Subtask 1 — Create the Left Widget

Objective

Create a custom widget named **Commercial Widget** and configure its HTML and CSS to provide an interactive visual element on the left side of the portal page.

Navigation Steps

1. Navigate to:

All → Service Portal → Service Portal Configuration

2. Select **Widget Editor**.
3. Click **Create a new widget**.
4. Enter the following details:
 - **Widget Name:** Commercial Widget
5. Click **Submit**.
6. Open the newly created widget.
7. Configure the required **HTML** code.
8. Configure the required **CSS** code.
9. Enable the required **Templates** for the widget.
10. Use the **Eye** icon to show or hide the preview while developing the widget.

Widget Behavior

The **Commercial Widget** is designed with a floating text/visual presentation.

The widget uses **cursor-based hover/movement behavior**, where the visual presentation responds as the cursor moves over the widget.

Expected Result

The **Commercial Widget** is successfully created and can be displayed on the portal page as an interactive visual component.

Subtask 2 — Create the Right Widget

Objective

Create a custom widget named **Link Redirect** to provide navigation from the portal to the **Raise Employee Issue Record Producer**.

Navigation Steps

1. Navigate to:

All → Service Portal → Service Portal Configuration

2. Select **Widget Editor**.
3. Click **Create a new widget**.

4. Enter:
 - **Widget Name:** Link Redirect
5. Click **Submit**.
6. Open the newly created widget.
7. Configure the required:
 - **HTML**
 - **CSS**
 - **Client Script**
8. Enable the required **Templates**.
9. Use the **Eye** icon to enable or hide the preview while developing the widget.

Expected Result

When the employee interacts with the **Link Redirect** widget, the widget redirects the user to the configured **Raise Employee Issue Record Producer**.

Subtask 3 — Add the Two Widgets to the Page

Objective

Add the **Commercial Widget** and **Link Redirect Widget** to the existing Employee Center Portal Page and position them appropriately.

Existing Page

The project uses an already-created page:

- **Page Name:** Employee Center Portal Page
-

Navigation Steps

1. Navigate to:

All → Service Portal → Service Portal Configuration

2. Open the **Designer**.
3. Search for the **Employee Center homepage/page**.
4. Select the required page.
5. Create a new **Container**.
6. Configure the container layout as:

6 × 6

7. Determine which widget should appear on each side of the container.
8. Search for the custom widgets:
 - **Commercial Widget**
 - **Link Redirect**
9. Drag and drop the widgets into their respective positions within the container.

Expected Result

After completing Phase 6:

- Two custom Service Portal widgets are created.
- **Commercial Widget** is configured with HTML and CSS.
- **Link Redirect Widget** is configured with HTML, CSS, and Client Script.
- The Link Redirect Widget can navigate users to the **Raise Employee Issue Record Producer** using its sys_id.
- Both widgets are added to the **Employee Center Portal Page**.
- The widgets are positioned using a **6 × 6 container layout**.
- Employees now have a customized portal interface for accessing the issue-raising functionality.

Phase 7 — Testing and Validation

Objective

This phase focuses on **end-to-end testing and validation** of the **Employee Raise Issue** application.

The purpose is to verify that all previously configured components work together as expected before the application is moved to a production environment.

The testing covers:

- Custom Employee Raise Issue table
 - UI Policies
 - Field dependencies
 - Record Producer
 - Service Portal
 - Custom Service Portal widgets
 - Record creation and field mapping
 - Field restrictions and validations
-

Description

The application is tested from an **end-user perspective** by accessing the Service Portal and raising an employee issue.

The testing verifies that employees can navigate to the portal, interact with the custom widgets, access the **Raise Employee Issue** Record Producer, enter the required information, and submit the issue successfully.

The form behavior configured through **UI Policies and dictionary configuration** is also validated.

Subtask 1 — Access and Validate the Service Portal

Navigation Steps

1. Open a **new browser tab**.
2. Enter the configured Service Portal URL:

https://dev307545.service-now.com/employee_request

3. Press **Enter**.
4. Verify that the **Requesting Portal / Employee Center portal** loads successfully.
5. Verify that the configured **image/logo and portal elements** are displayed correctly.

Expected Result

The Service Portal loads successfully and displays the configured branding, image, and portal components.

Subtask 2 — Validate the Custom Widgets

Navigation Steps

1. Scroll down through the portal page.
2. Locate the two custom widgets created in **Phase 6**:
 - **Commercial Widget**
 - **Link Redirect Widget**
3. Move the **cursor** over the widgets.
4. Verify that the configured floating/hover interaction works as expected.
5. Click the **Link** button provided by the Link Redirect Widget.

Expected Result

The widgets are displayed in their configured positions and their interactive behavior works correctly.

The **Link Redirect Widget** redirects the employee to the **Raise Employee Issue Record Producer**.

Subtask 3 — Test the Record Producer and Form Behavior

Record Producer Validation

After clicking the Link button:

1. Verify that the **Raise Employee Issue** Record Producer opens.
2. Verify that the employee can enter the required issue information.
3. Check the available fields:
 - Requester
 - Category
 - Subcategory
 - State
 - Short Description
4. Verify the **Category → Subcategory** dependency.
5. Enter the required information.
6. Click **Submit**.

Expected Result

The Record Producer successfully processes the employee's request and creates a corresponding record in the **Employee Raise Issue** custom table.

UI Policy and Field Restriction Validation

During testing, verify the behavior configured in **Phase 3**.

Test 1 — Short Description

Set:

State → New

Verify that:

Short Description → Mandatory

The user should be required to provide a short description when the state is **New**.

Test 2 — Number Field

Verify that the **Number** field is read-only.

The user should not be able to manually modify the automatically generated issue number.

Test 3 — Category and Subcategory

Verify that the Subcategory choices correspond to the selected Category.

Category Expected Subcategory

Network VPN

Hardware Laptop

Software Server

Access Forgot Password

This confirms that the configured **field dependency** is functioning correctly.

Phase 7 — Expected Outcome

After completing testing and validation, the **Employee Raise Issue** application should successfully support the complete employee issue-raising process:

Employee → Service Portal → Custom Widget → Record Producer → Validation → Employee Raise Issue Table

The testing confirms that the configurations implemented throughout **Phases 1–6** work together correctly and that the application is ready for further administrative review and, after appropriate approvals, production deployment.

Phase 8 — Conclusion

Project Conclusion

The **Employee Raise Issue — Record Producer & Service Portal Integration** project successfully delivers a **centralized, user-friendly, and structured self-service solution** that enables employees to raise workplace-related issues through the ServiceNow Service Portal.

The implementation integrates the **Employee Center scoped application, custom Employee Raise Issue table, UI Policies, field dependencies, Record Producer, custom Service Portal widgets, and dedicated Service Portal** into a single end-to-end solution.

The solution provides the following key benefits:

- **Standardized and accurate issue submission** through structured forms and predefined fields.
- **Business-rule enforcement** through UI Policies and form-level restrictions.
- **Controlled category and subcategory selection** using field dependencies.
- **Seamless integration between the Service Portal and backend custom table.**
- **Improved issue visibility and tracking** through structured issue records.

- **Reduced manual effort** for HR and support teams.
- **Reduced data inconsistencies** by enforcing standardized information capture.
- **Improved employee experience** through an easy-to-use self-service interface.
- **Centralized issue management** within the ServiceNow platform.

The **end-to-end testing and validation** performed in Phase 7 confirmed that the major components of the application function as expected, including the Service Portal, custom widgets, Record Producer, field mappings, UI Policies, and field dependencies.

Overall, the implementation provides a scalable foundation for managing employee workplace issues and can be further enhanced with features such as **automated assignment, notifications, SLA management, approval workflows, reporting, dashboards, and advanced business rules**.

Final Outcome:

The project transforms an informal, email-based employee issue-raising process into a **standardized ServiceNow self-service workflow**, improving data quality, process efficiency, issue visibility, and the overall employee experience.